

Introduction to Algorithm

기초 알고리즘

Greedy 알고리즘 개요

Greedy

우리는 앞에서 다양한 알고리즘들을 공부하면서,
각 상황별로 최적의 답을 내놓으려고 노력했음



그렇다면, 잠깐 컴퓨터에서 벗어나
현실의 문제를 살펴보도록 하자



그리디 (Greedy)

B

A

A라는 지점에서 B라는 지점으로
최대한 빠르게 이동하려고 함

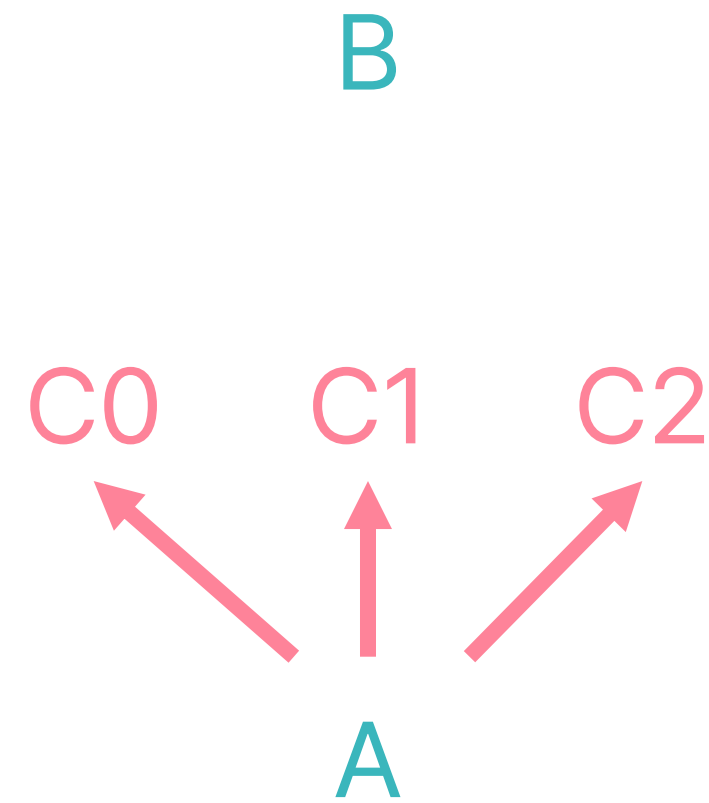


그리디 (Greedy)

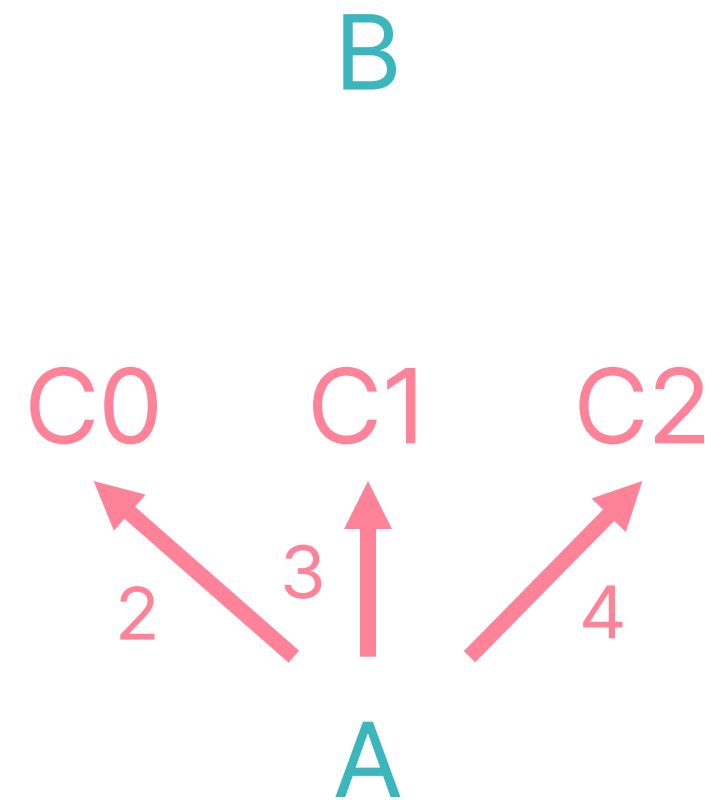
B

A

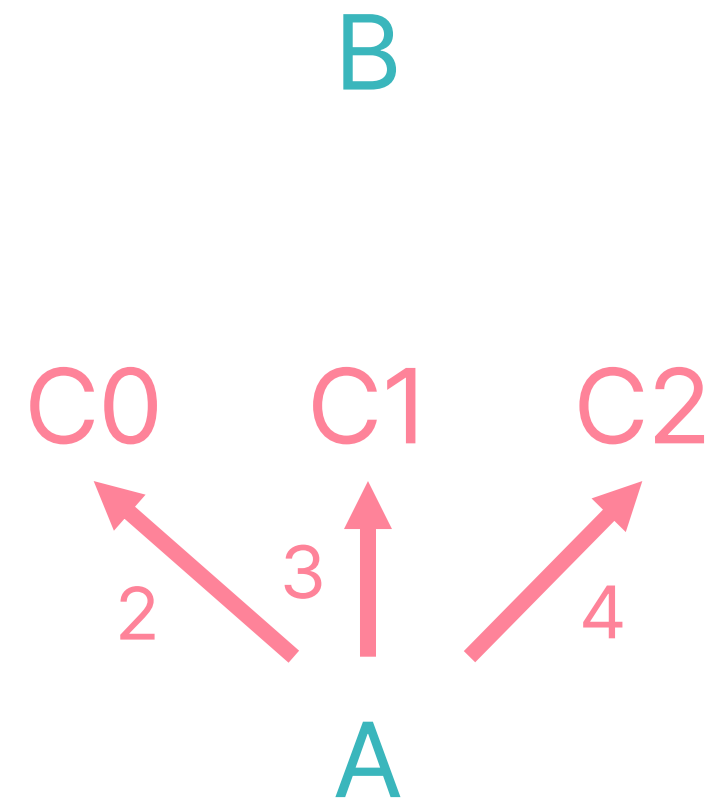
그런데 핸드폰을 두고 와서 어떻게 가야 빨리 갈 수 있을지
모르겠다면?



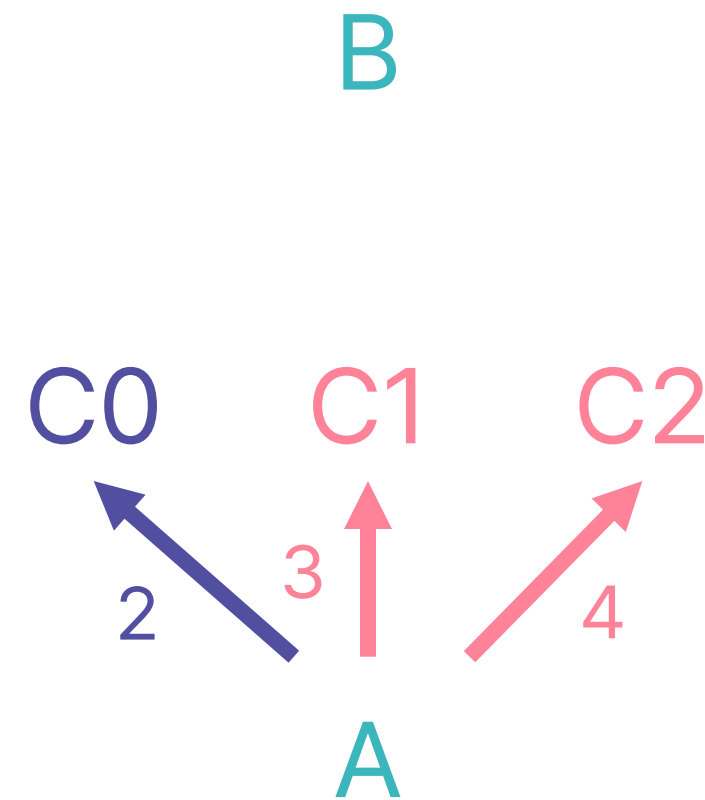
앞을 보니, 서로 다른 세 지점으로
이동할 수 있는 길이 있다면?



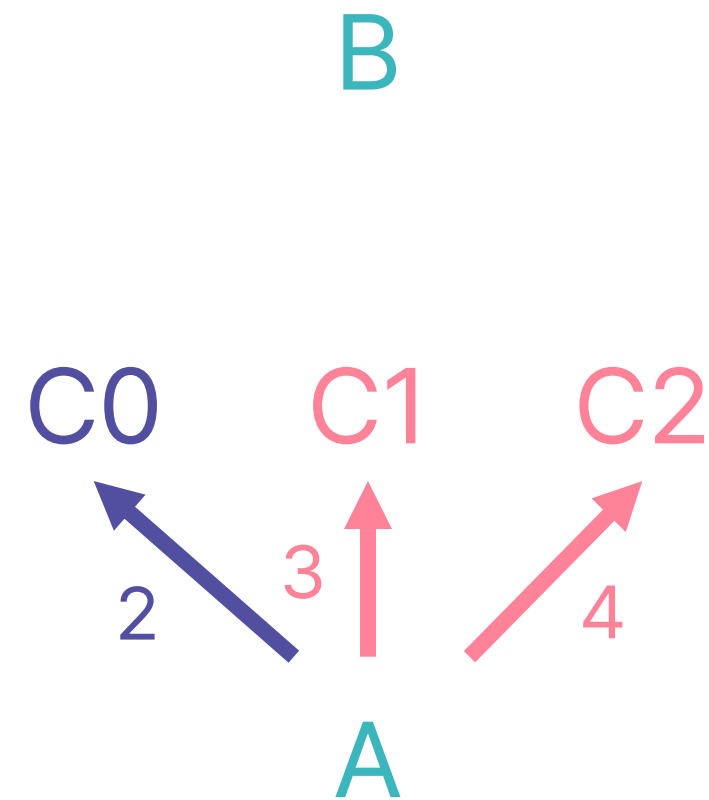
각각의 지점으로 이동하는 길이는 서로 다름
이 상황에서, 우리는 어떤 길을 골라야 할까?



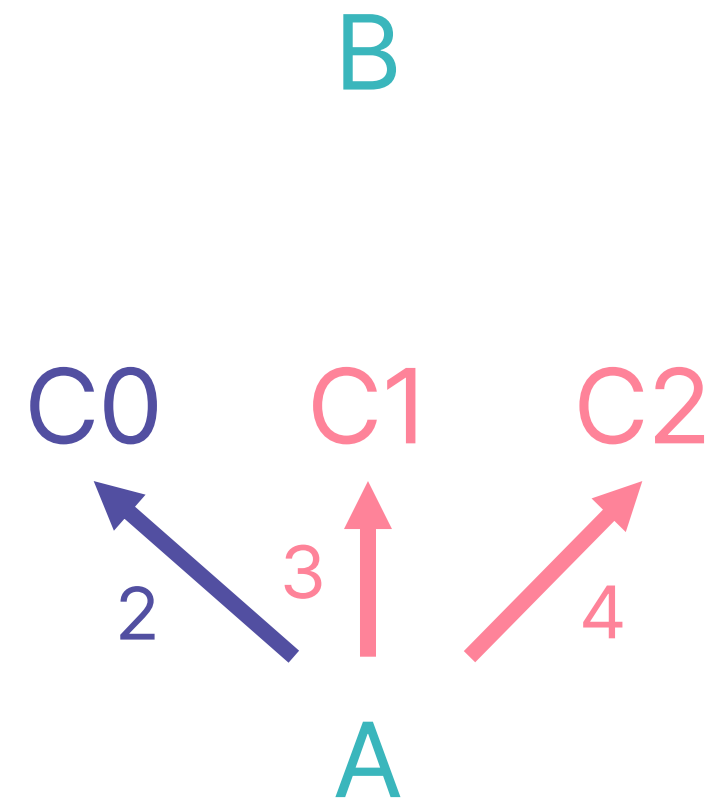
우리가 현재 갖고 있는 정보가 너무 한정적이기 때문에,
어떤 선택을 해도 최적의 선택인지 알 수 없음



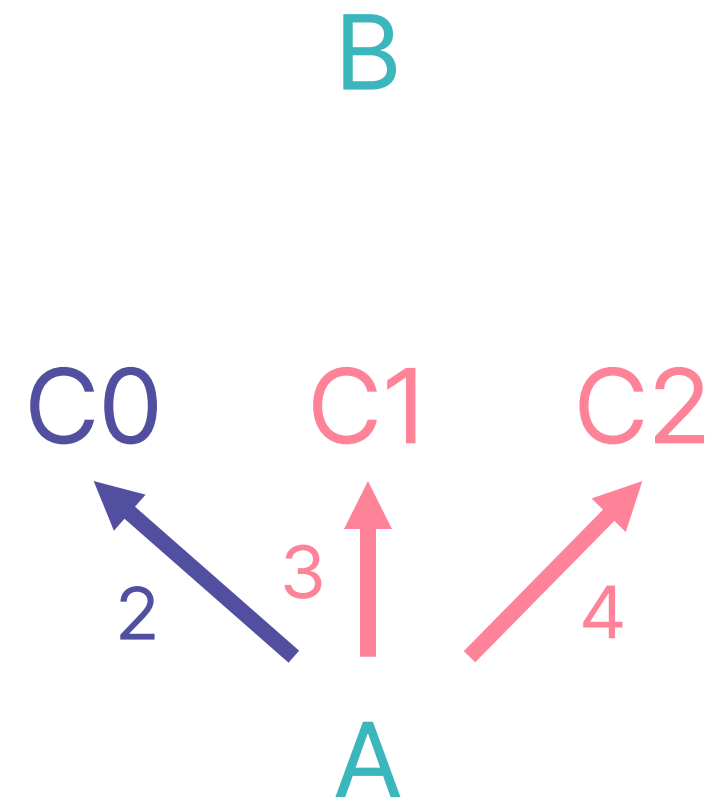
그렇지만, 현재 시점에서의 최적의 선택지를 고르라면,
가장 거리가 짧은 C0으로 이동하는 게 될 것



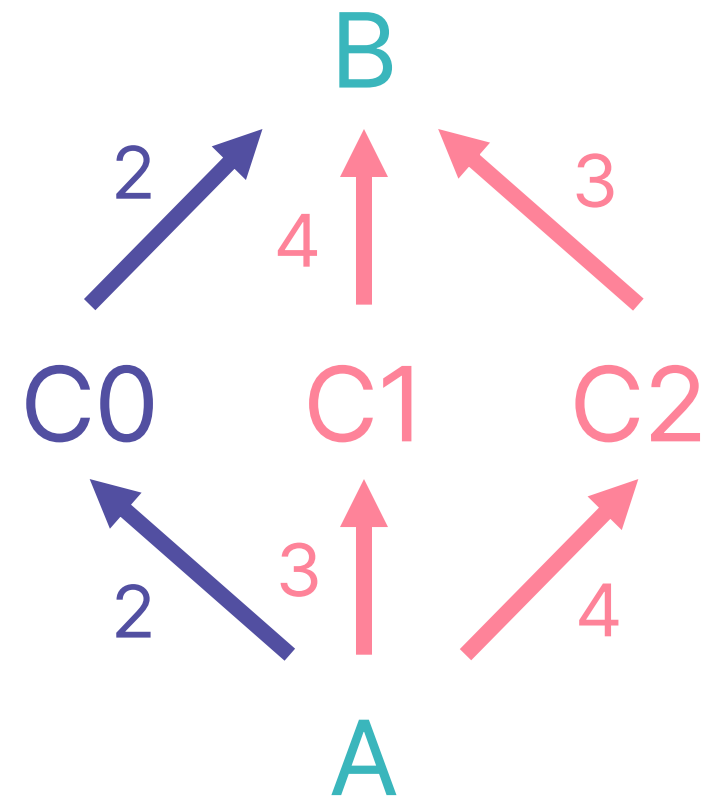
이런 방식으로, 현재 시점에서의 최적의 선택을 반복해서
답을 얻는 방식이 바로 **그리디** 알고리즘



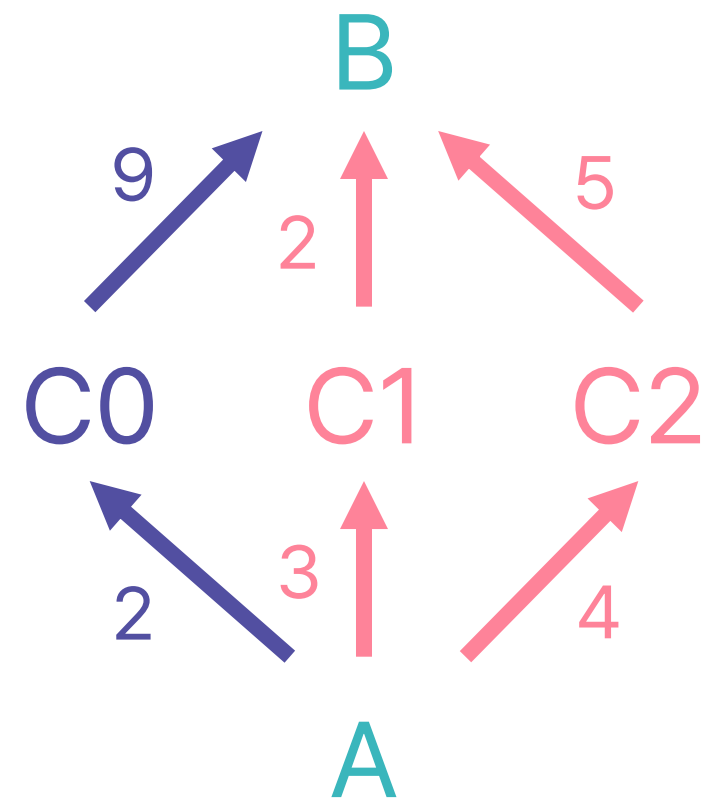
그리디 (Greedy) 의 뜻은 '탐욕적인' 으로,
탐욕적으로 현재 가장 이득인 선택지를 고른다는 의미



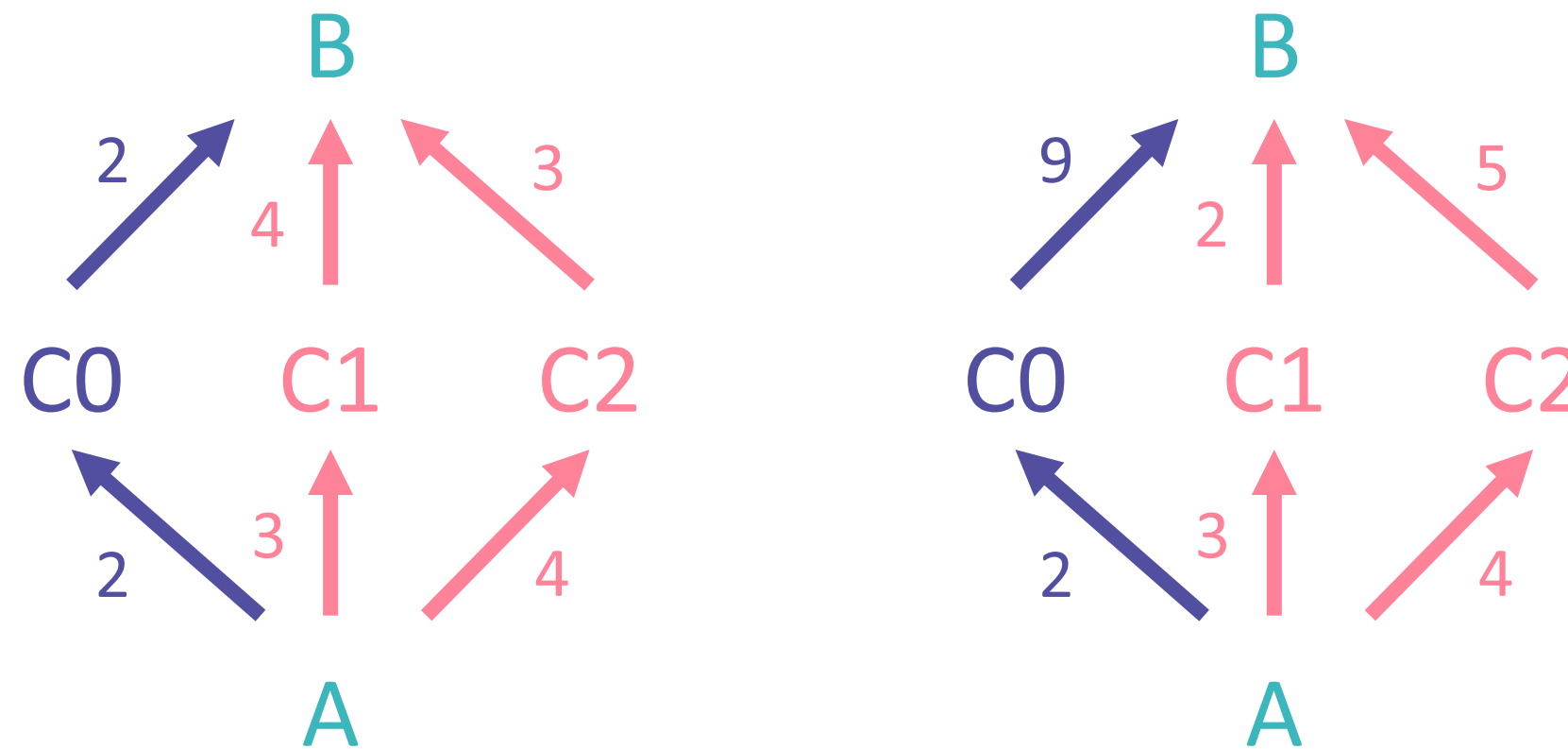
그리디 (Greedy)는 현재 시점에서 최적해를 고르는 것을
반복하기 때문에, 전체 최적해가 될 수도, 아닐 수도 있음



위 예시에서, 모든 경로가 다음과 같다면
우리의 선택은 최적의 선택이 될 것



그렇지만, 위와 같이 변경되면 최적해가 아니게 됨

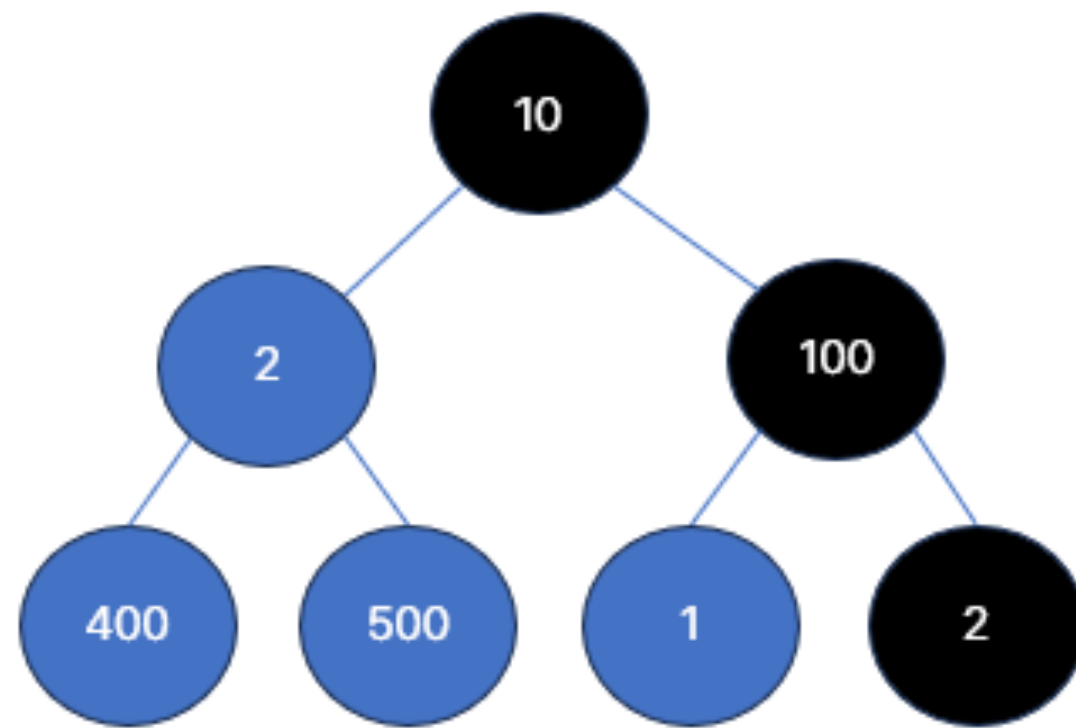


문제에 따라 그리디 (Greedy)가 최적해가 되는 경우가 있고,
그렇지 않은 경우가 있음

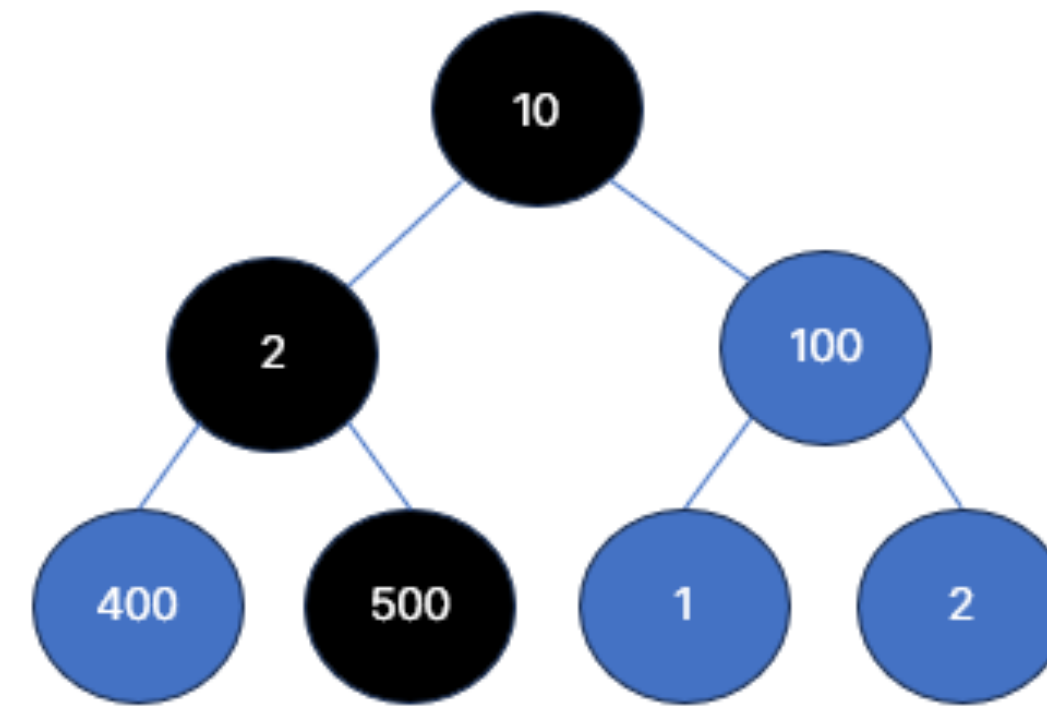


그리디 (Greedy)

- 선택의 순간마다, 당장 눈앞에 보이는 최적의 상황만을 쫓아 최종적인 해답에 도달하는 방법
 - 탐욕스러운(Greedy) 기법
- 순간마다 하는 선택은 그 순간에 대해 지역적으로는 최적
 - 그 선택들이 모이면 최종적(전역적)으로도 최적일까?



그리디



최적



그리디 (Greedy)

예시

1000원짜리 커피를 500원짜리 동전과 100원짜리 동전만 사용하여 계산하려고 한다.
동전을 가장 적게 사용하여 계산하려고 할 때, 필요한 동전의 최소 개수는? (단, 동전은 무수히 많다)



100원짜리 5개 사용하는 것보다
500원짜리 동전 하나를 사용하는 것이 **항상 최적**
→ 그리디 알고리즘으로 해결 가능



입력 예시

1330 4

500

100

50

10

출력 예시

8

첫 줄에 만들고 싶은 금액과 동전의 종류의 수가 나오고,

이어서 각 동전의 금액이 나옴



입력 예시

```
1330 4
500
100
50
10
```

출력 예시

```
8
```

각각의 동전에는 하나의 조건이 있는데,
입력된 동전의 금액은 그 다음 줄에 입력된 금액과 배수 관계라는 것



입력 예시

1330 4

500

100

50

10

출력 예시

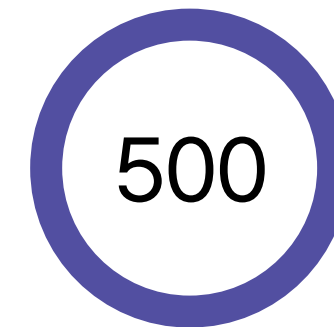
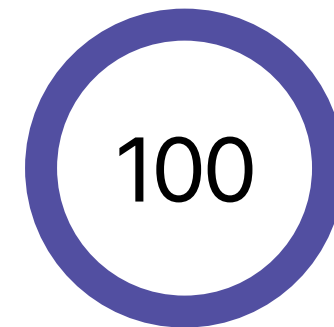
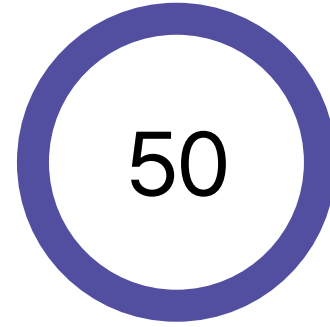
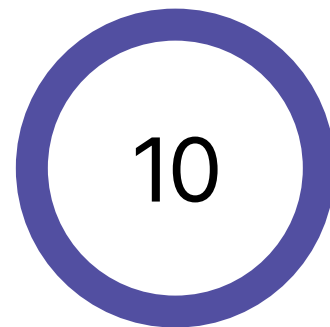
8

우리의 목표는, 최소한의 동전을 사용해서
주어진 금액을 만드는 것



동전의 개수: 0개

남은 금액: 1330원

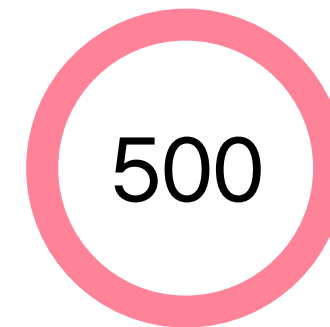
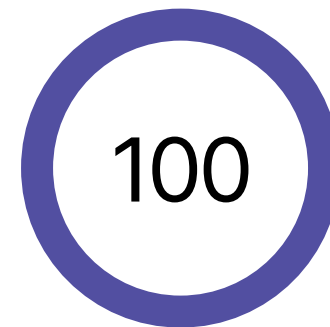
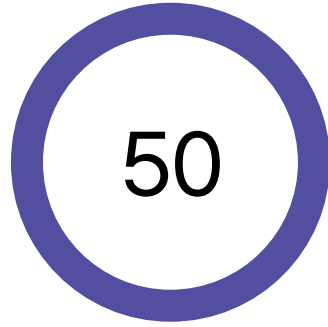
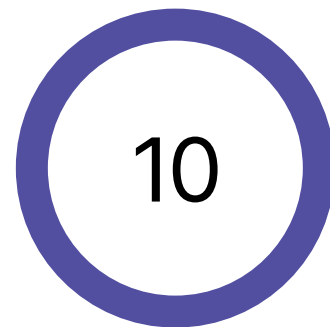


어떤 동전을 골라야 할까?



동전의 개수: 0개

남은 금액: 1330원



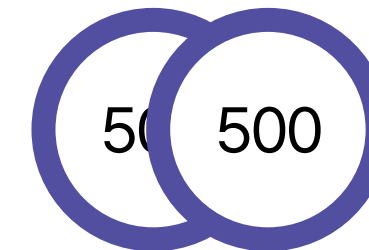
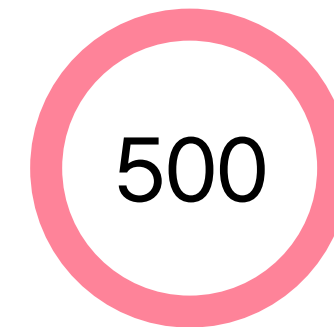
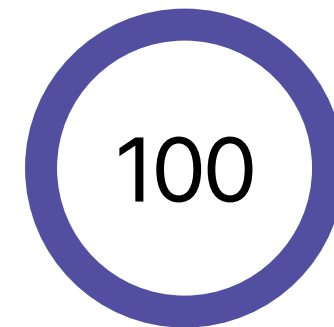
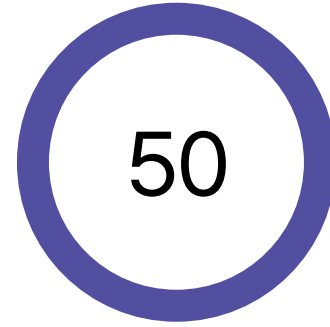
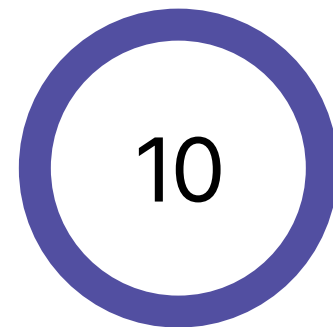
특별히 떠오르는 선택지가 없다면,
우선 가장 큰 동전을 선택하는 것은 어떨까?



[예제] 저금통

동전의 개수: 2개

남은 금액: 330원



가장 큰 동전을 가져갈 수 있을 만큼

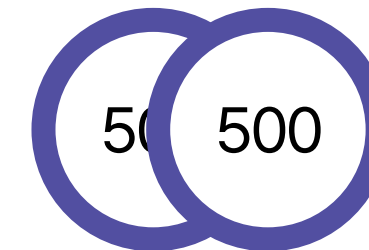
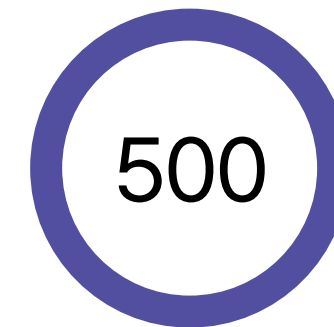
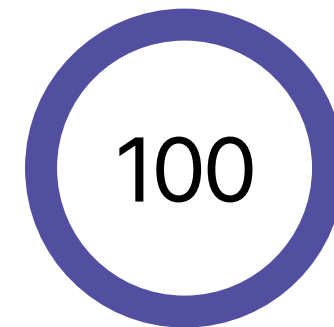
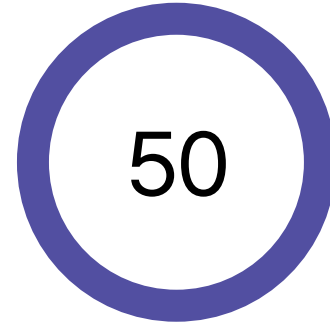
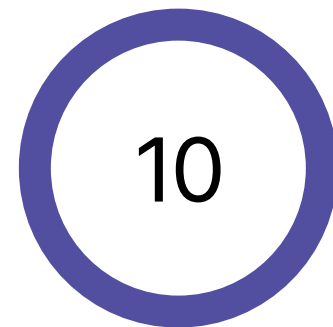
최대한 가져가 본다면?



[예제] 저금통

동전의 개수: 2개

남은 금액: 330원

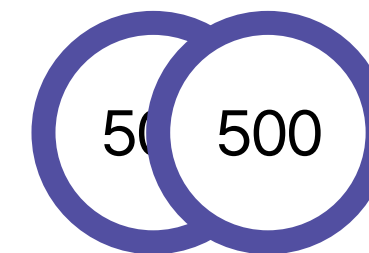
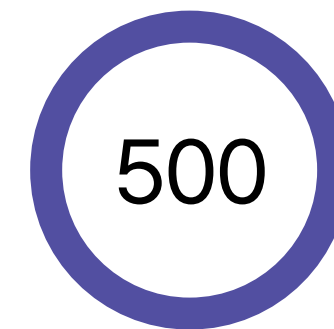
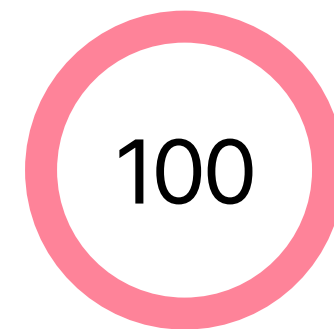
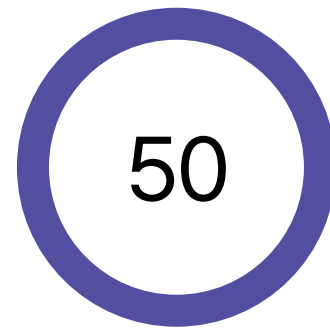
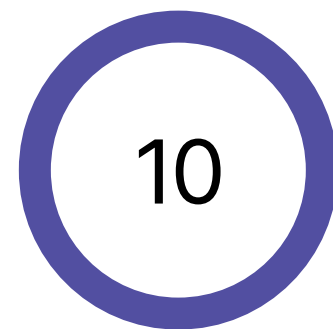


이제 500원은 더 이상 가져갈 수 없음



동전의 개수: 2개

남은 금액: 330원



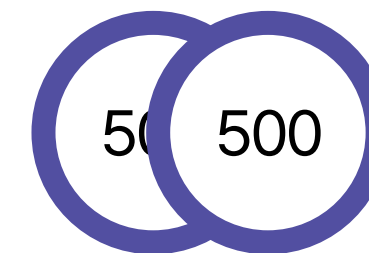
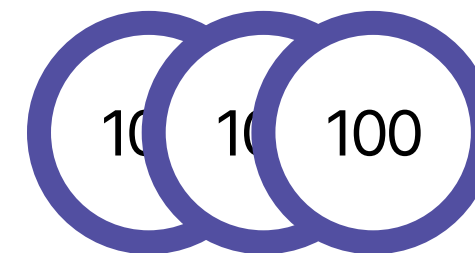
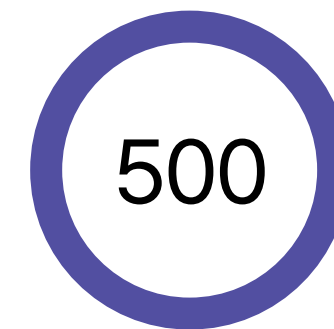
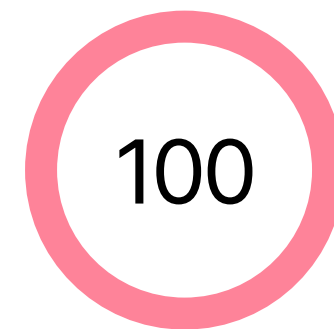
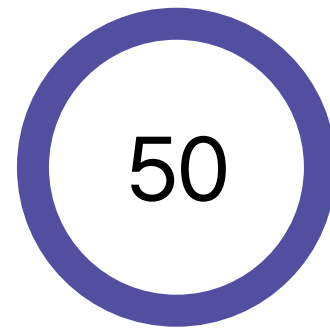
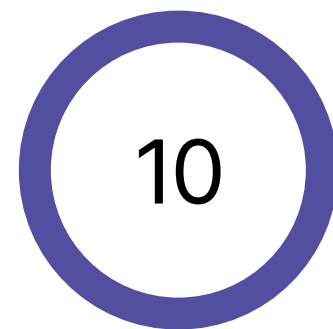
그렇다면 이번엔 100원을 가져가 보도록 하자



[예제] 저금통

동전의 개수: 5개

남은 금액: 30원



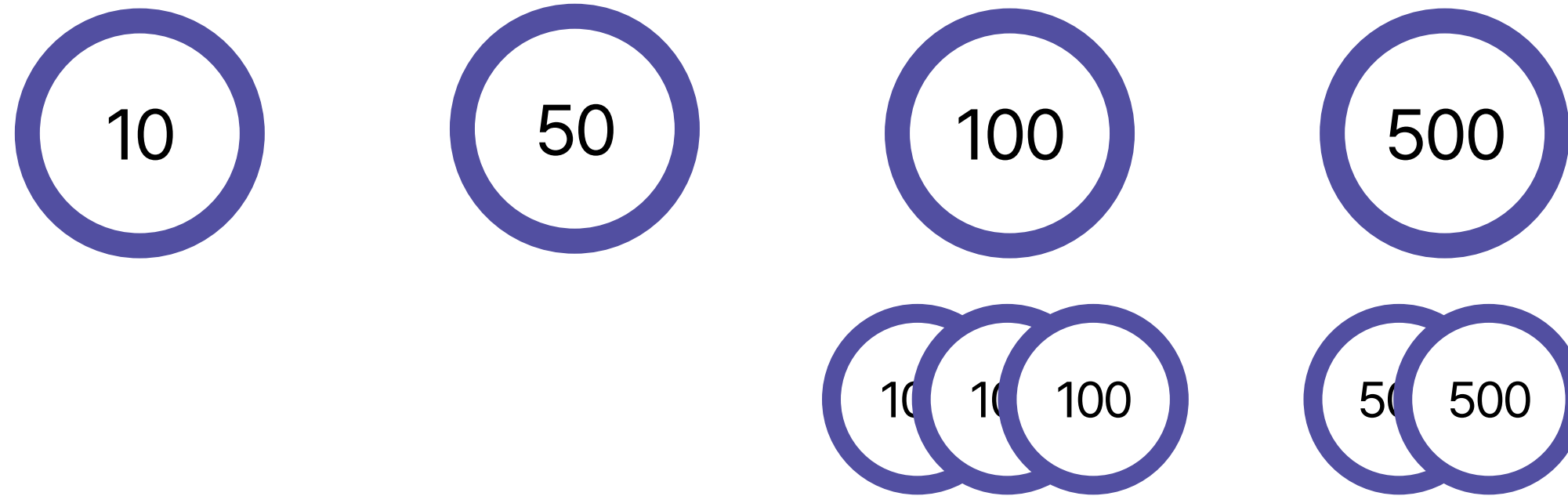
330원이 남았으니, 100원 동전을 3개 가져갈 수 있음



[예제] 저금통

동전의 개수: 5개

남은 금액: 30원



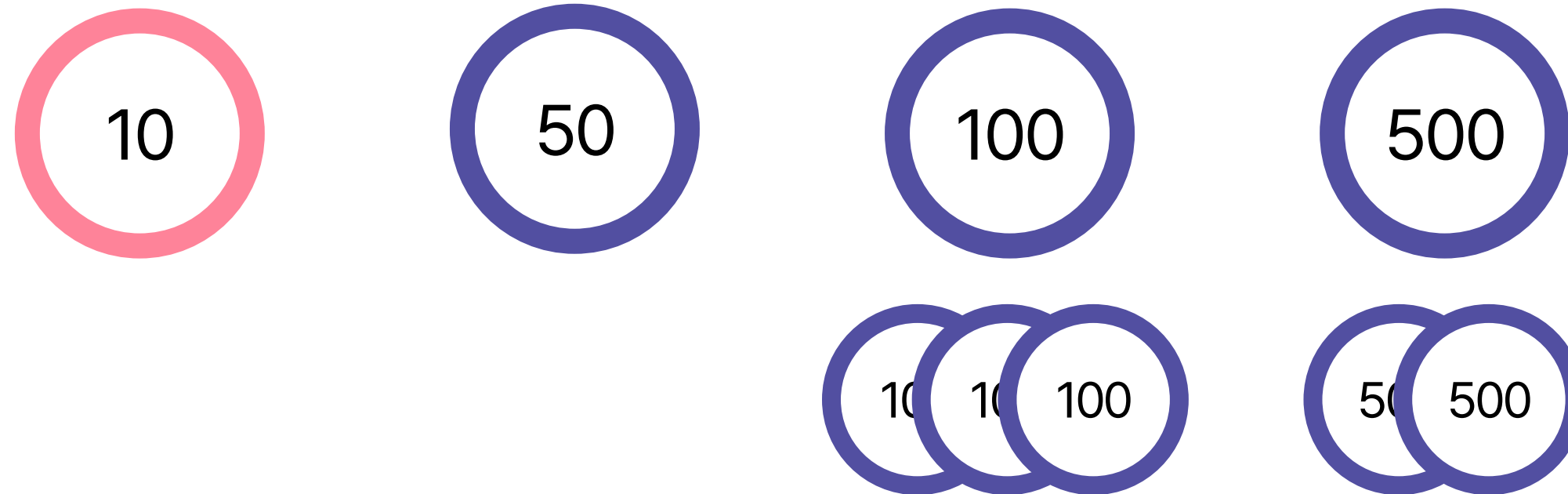
이어서 50원을 가져가고 싶지만,
남은 금액이 50원이 안 되므로 넘어감



[예제] 저금통

동전의 개수: 5개

남은 금액: 30원



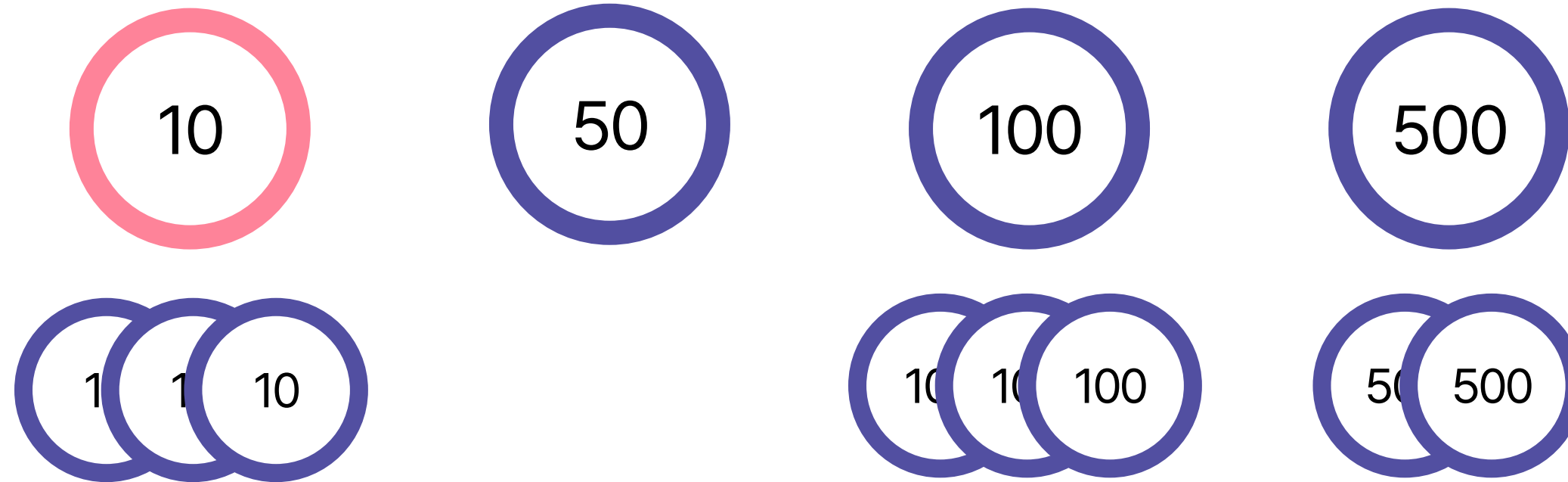
그렇다면, 마지막으로 10원 짜리를 가져간다면?



[예제] 저금통

동전의 개수: 8개

남은 금액: 0원



3개의 동전을 가져갈 수 있고,

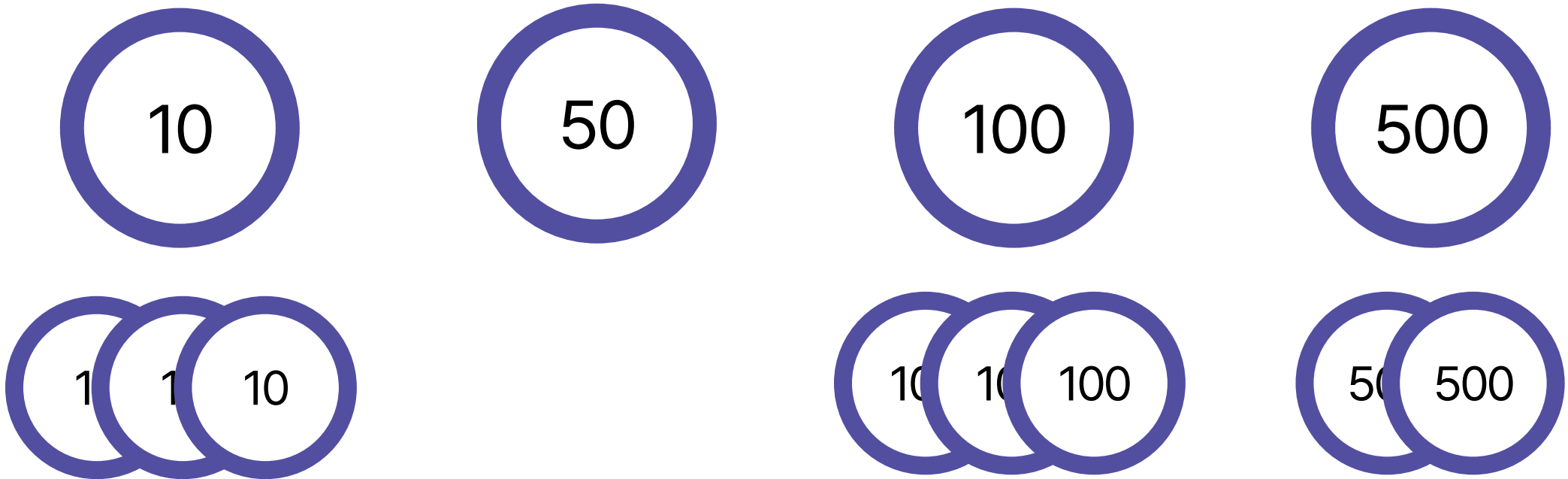
동전 교환이 끝남



[예제] 저금통

동전의 개수: 8개

남은 금액: 0원

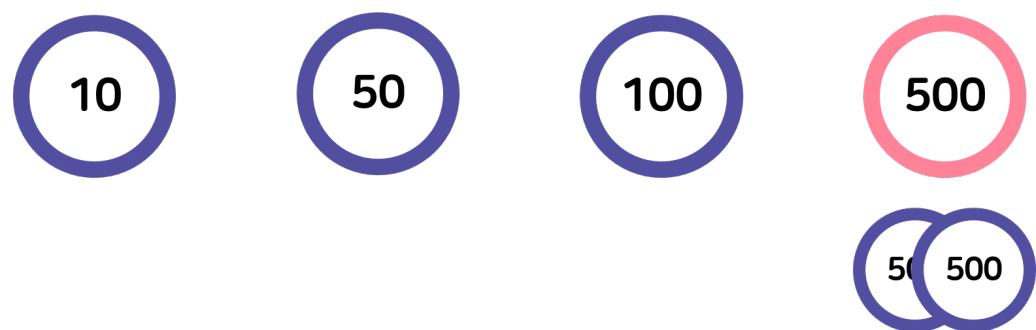


이 풀이가 정답

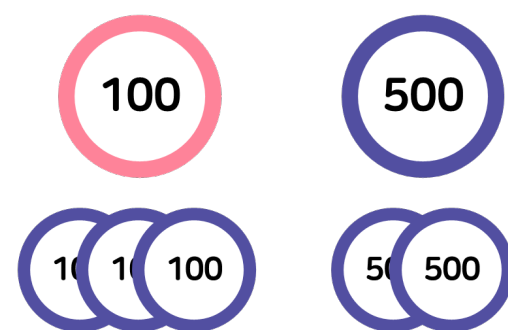


[예제] 저금통

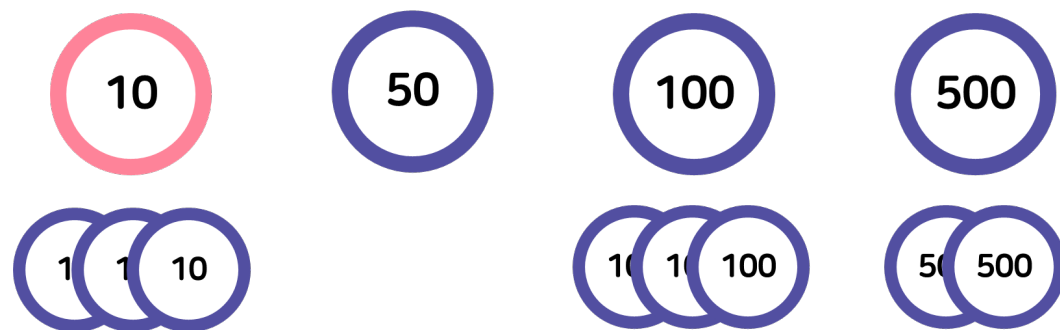
동전의 개수: 2개
남은 금액: 330원



동전의 개수: 5개
남은 금액: 30원



동전의 개수: 8개
남은 금액: 0원



우리는 순간 순간에 교환할 수 있는
가장 큰 값을 선택했음



[예제] 저금통



그리디 알고리즘은 지금 그 **순간**에 선택할 수 있는

최적의 선택을 하는 알고리즘이라는 것

Greedy의 조건



그렇다면, 모든 문제를 그리디 알고리즘으로 풀 수 있을까?



그리디의 조건

입력 예시

1330 4

500

100

50

10

출력 예시

8

앞에서 해결했던 동전 문제를 다시 보면,
동전의 금액은 그 다음 줄에 입력된 금액과
배수 관계 라는 조건이 있었음



그리디의 조건

입력 예시

1330 4

500

100

50

10

출력 예시

8

만약, 이 조건이 없다면

그리디 알고리즘의 풀이가 성립할까?



그리디의 조건

동전의 개수: 0개

남은 금액: 1200원

100

400

600

900

앞에서 활용했던 그리디 풀이를 적용해보도록 하자



그리디의 조건

동전의 개수: 1개

남은 금액: 300원

100

400

600

900

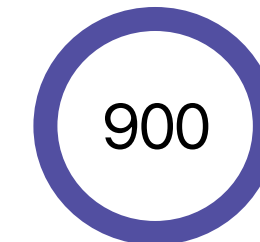
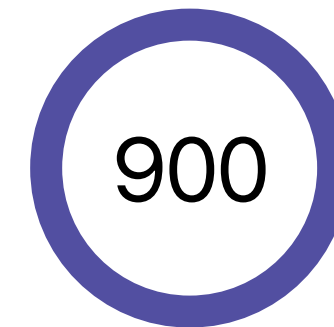
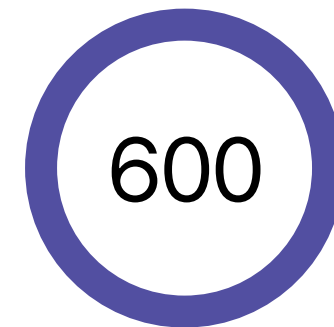
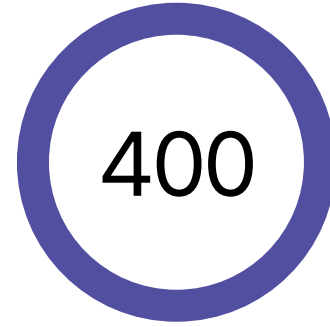
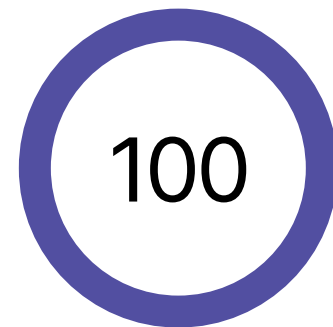
900

먼저 900원을 가져갈 수 있음



동전의 개수: 1개

남은 금액: 300원



300원이 남았기 때문에 400원/600원 동전은

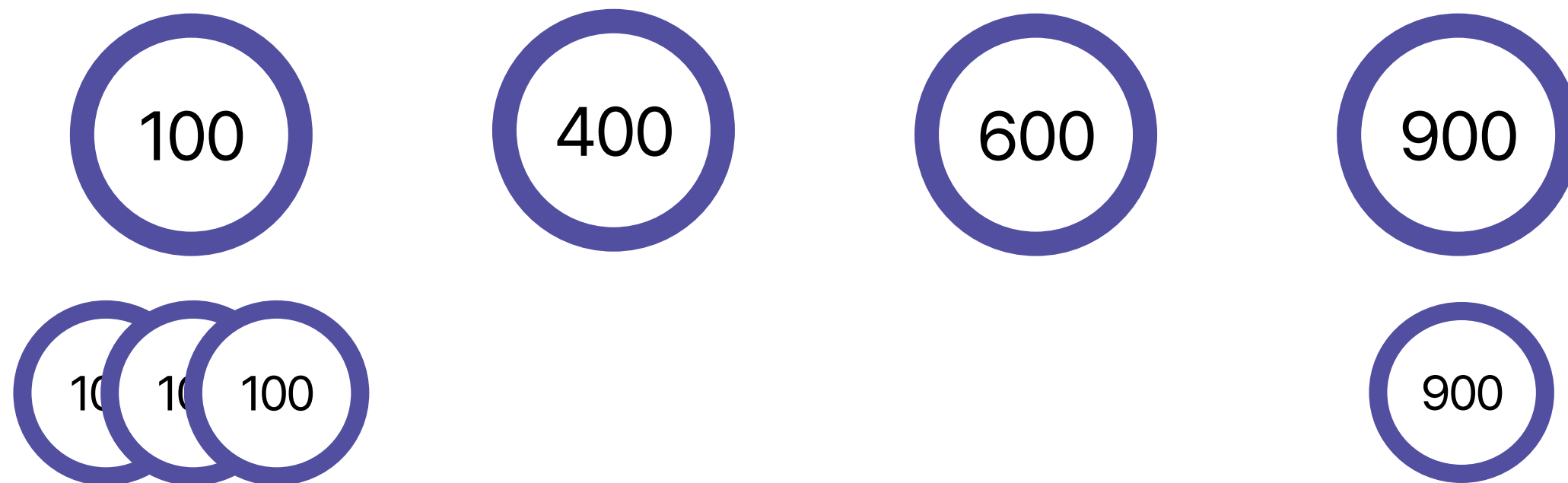
더 이상 가져갈 수 없음



그리디의 조건

동전의 개수: 4개

남은 금액: 0원



결국, 100원 동전 3개를 가져가고

동전 교환이 끝나게 됨



그리디의 조건

100

400

600

900

그런데, 다시 문제를 살펴보면...

600원 동전 2개를 가져가면 단 2개의 동전으로 교환이 끝남



그리디의 조건

100

400

600

900

즉, 그리디 알고리즘으로 문제가 해결되지 않음



그렇다면, 어떤 문제를 그리디로 해결할 수 있을까?

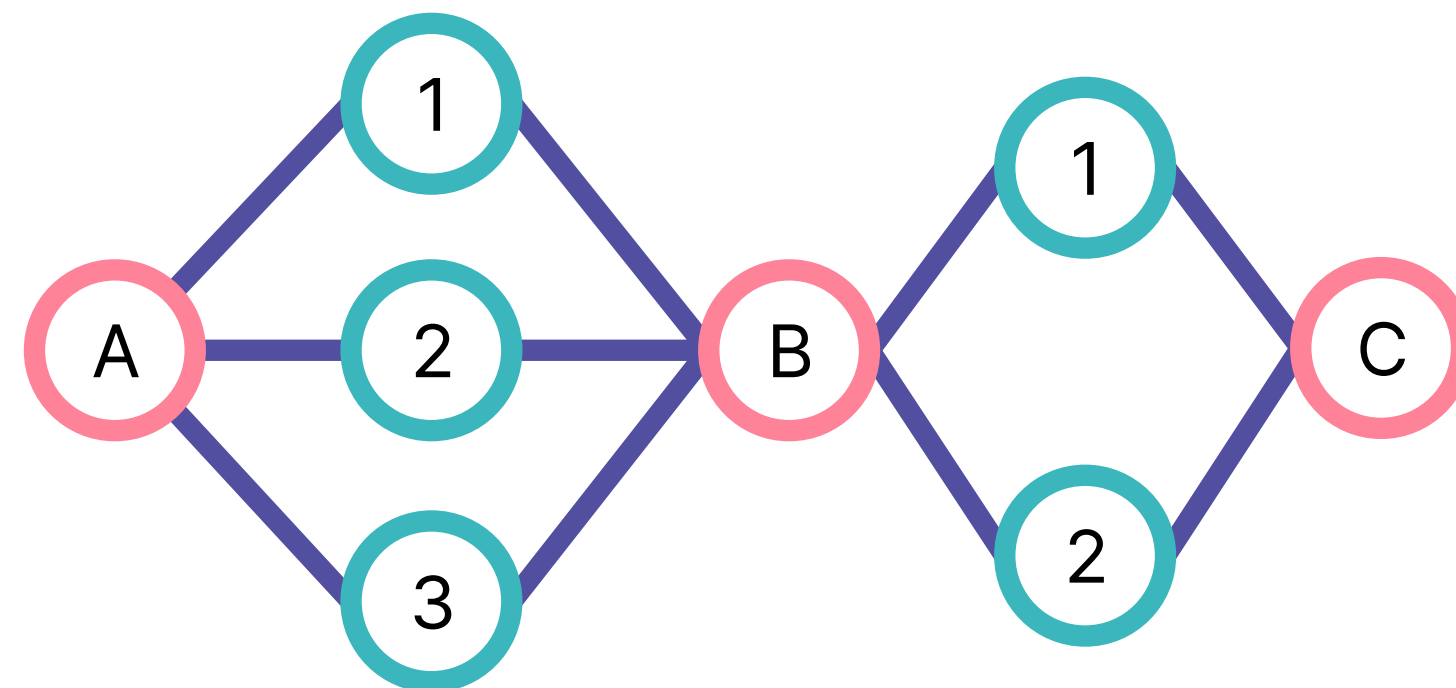


그리디 알고리즘으로 문제가 해결되는 조건은,
최적 부분 구조와 탐욕적 선택 조건을 만족하는 것



최적 부분 구조란,

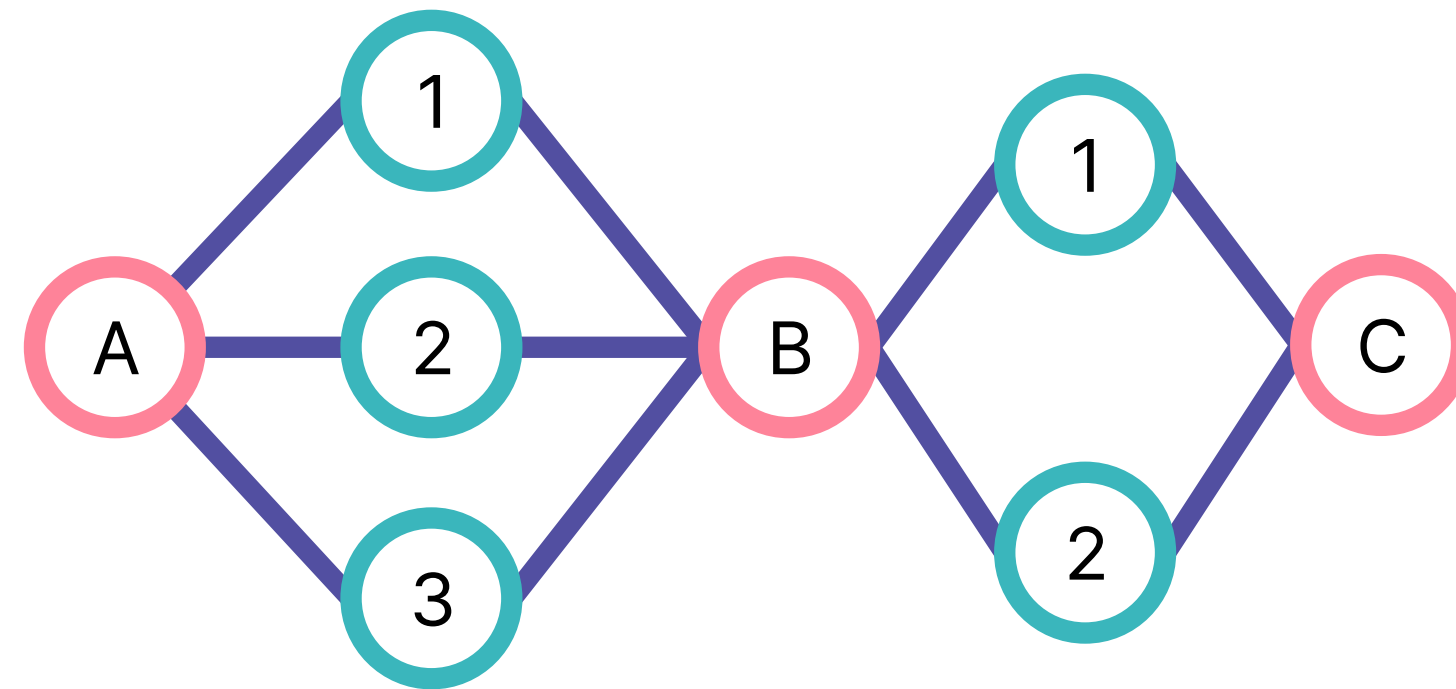
문제를 부분 문제로 쪼갤 수 있는 구조를 의미함



다음과 같은 길이 있다고 가정해보자



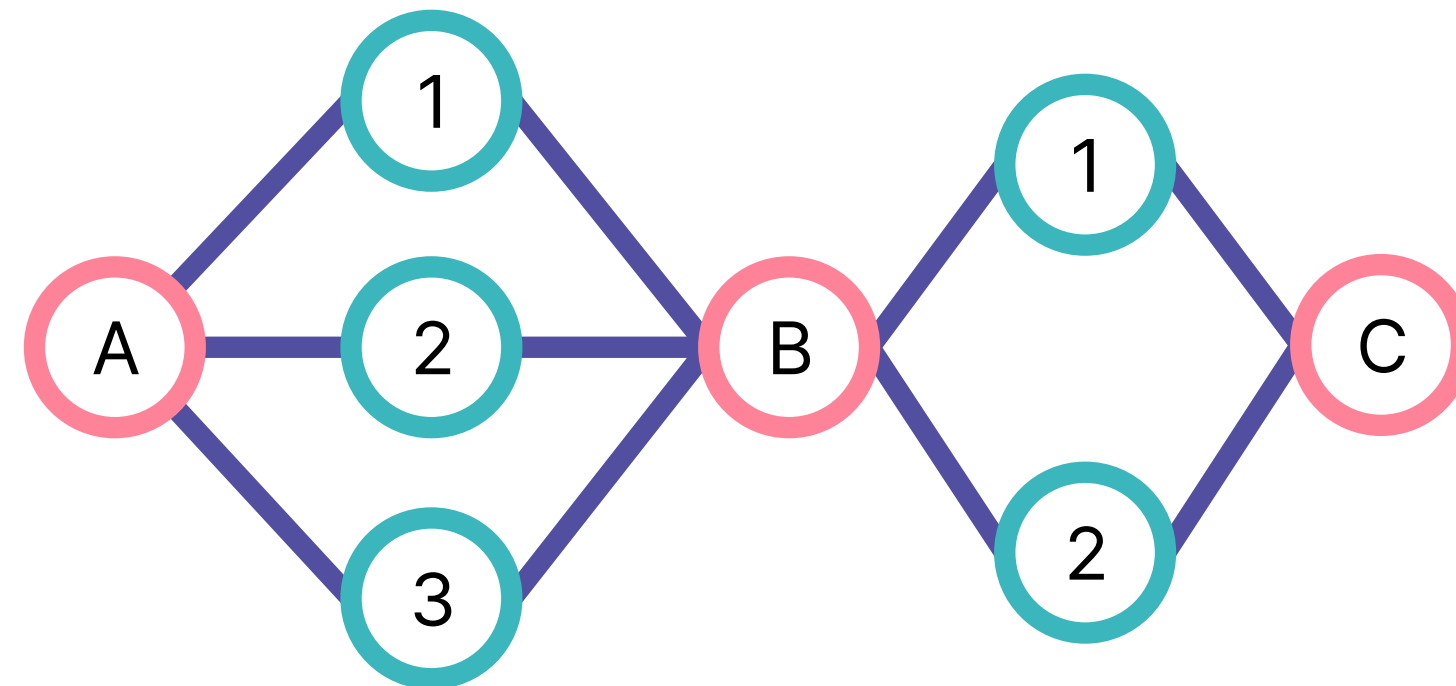
최적 부분 구조



A에서 C로 가는 최단거리를 구하고 싶음



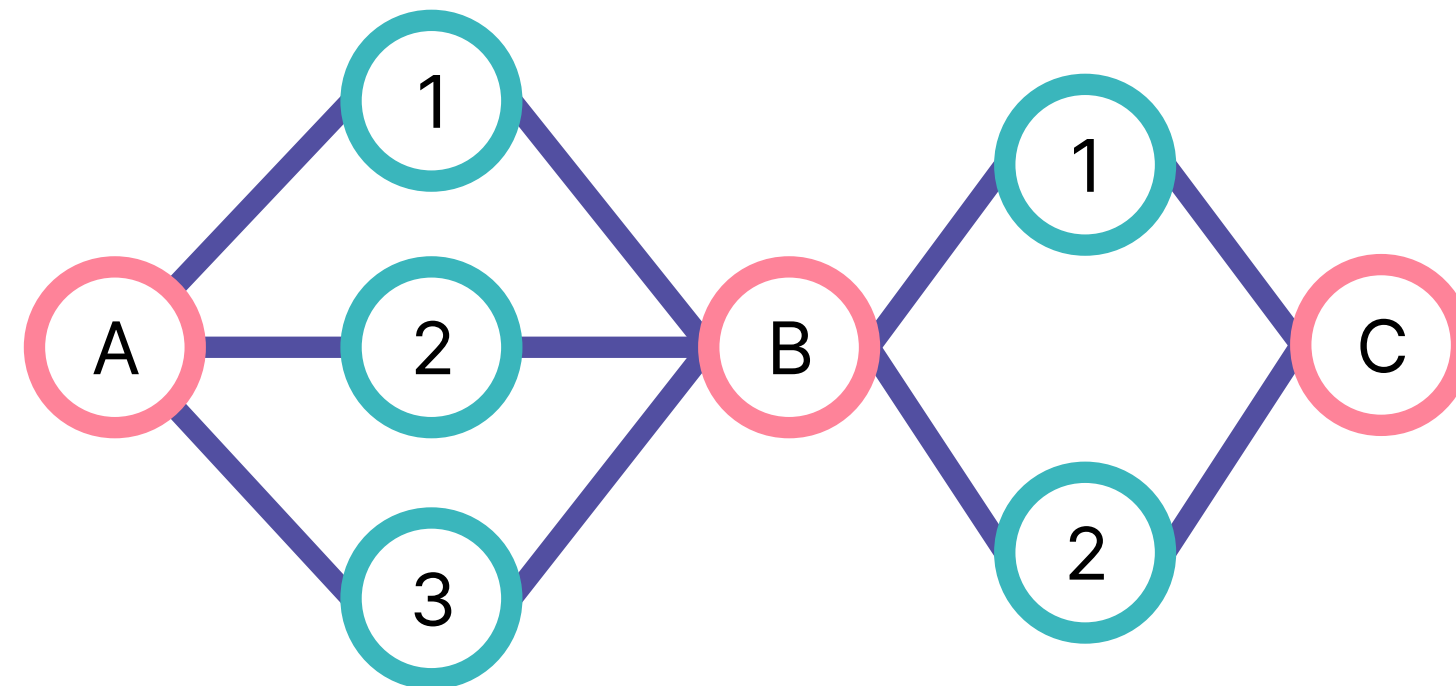
최적 부분 구조



그런데, 잘 살펴보면 모든 경로는 반드시 B를 지나고 있음

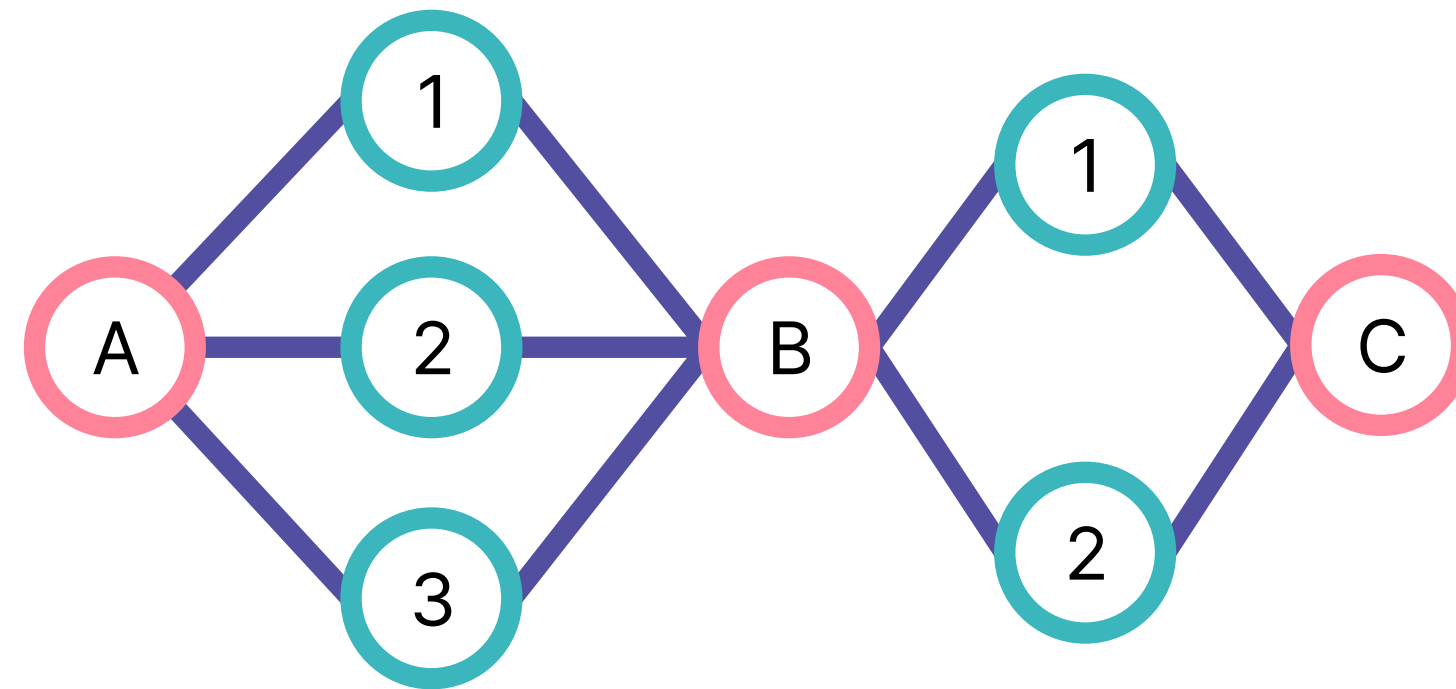


최적 부분 구조



즉, (A - C 의 최단거리) =

(A - B 의 최단거리) + (B - C 의 최단거리) 가 성립



이런 식으로, 문제를 부분 문제로 쪼갤 수 있으면
이 문제가 **최적 부분 구조**를 띄고 있다고 할 수 있음



탐욕적 선택 조건이란, 순간의 최적 선택이
전체 문제의 최적해에 반드시 포함된다는 것



탐욕적 선택 조건

동전의 개수: 0개

남은 금액: 1330원

10

50

100

500

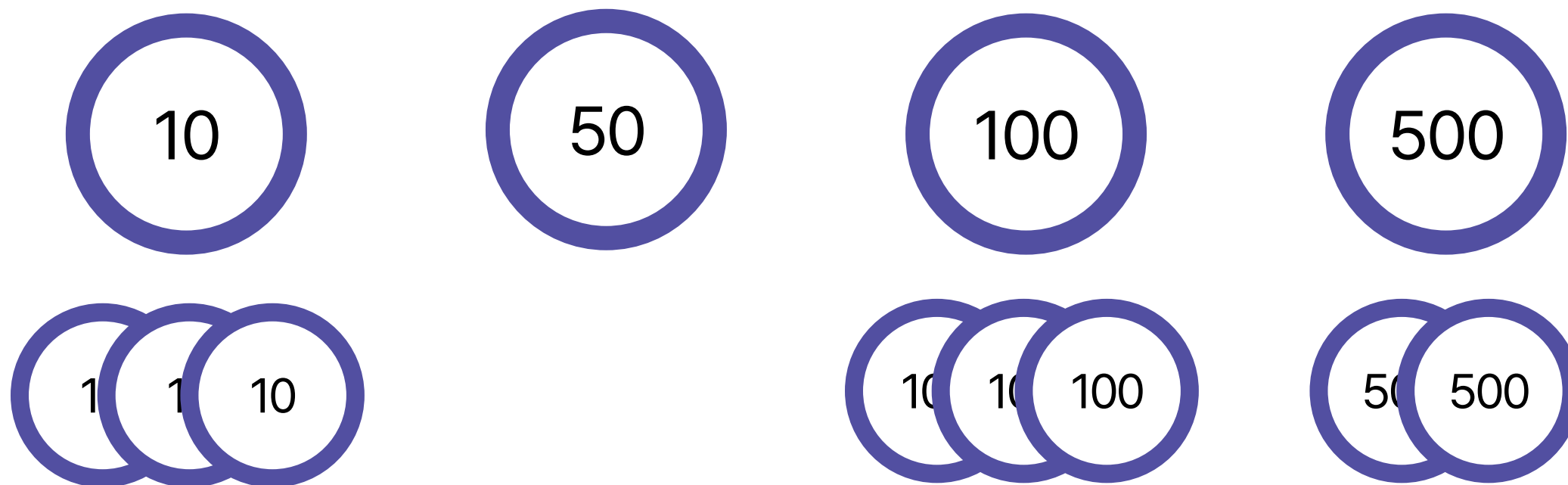
동전 문제를 다시 살펴보도록 하자



탐욕적 선택 조건

동전의 개수: 8개

남은 금액: 0원



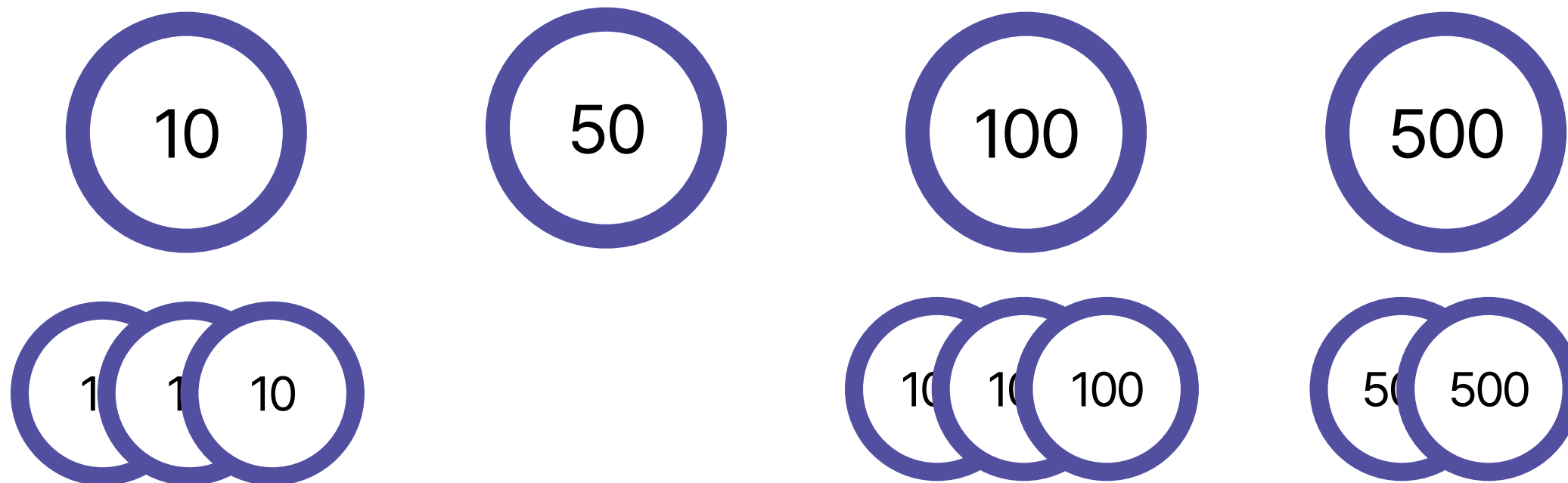
잘 생각해보면, 500원을 2개 선택 했고,
다른 동전도 동일한 방법으로 선택 했음



탐욕적 선택 조건

동전의 개수: 8개

남은 금액: 0원



그렇다면, 8개를 사용해서 1330원을 만들 수 있는

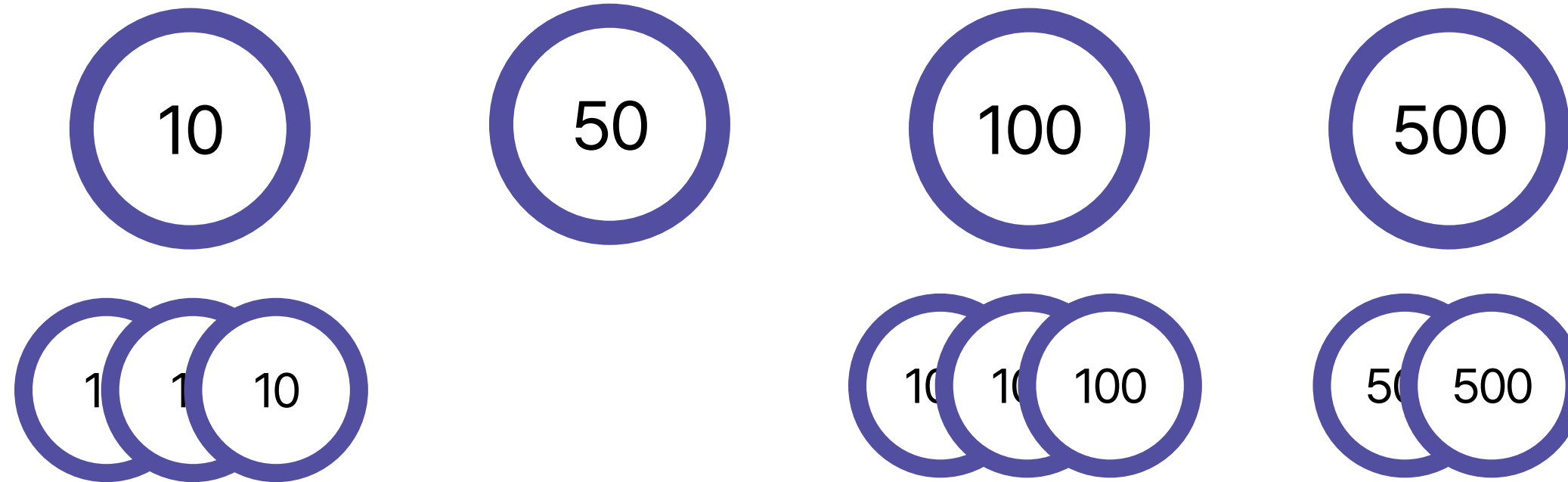
다른 방법은 없을까?



탐욕적 선택 조건

동전의 개수: 8개

남은 금액: 0원



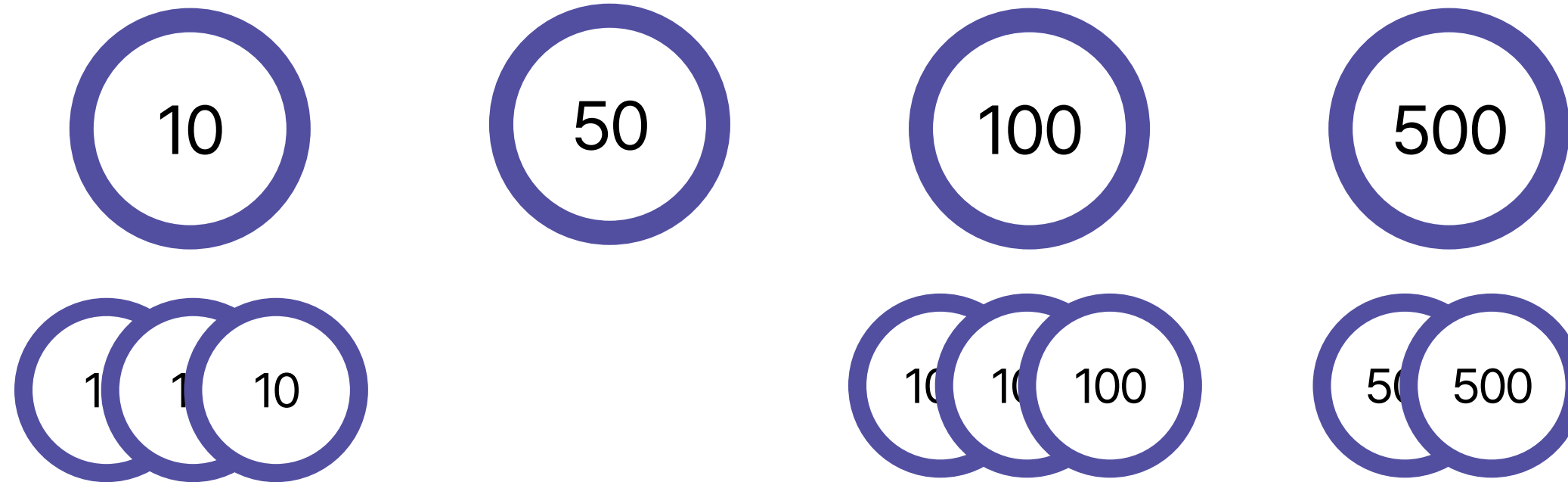
결과적으로 말하자면, 다른 조합은 불가능함



탐욕적 선택 조건

동전의 개수: 8개

남은 금액: 0원

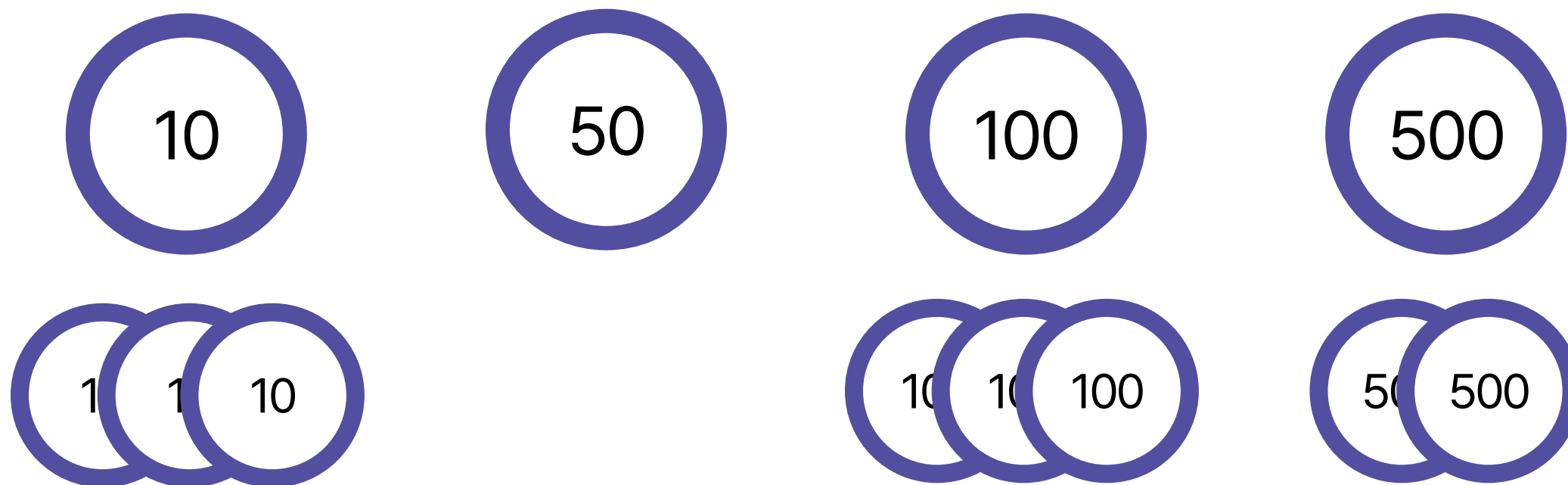


만약 다른 조합이 1개라도 있었다면, 문제를 푸는 과정에서의
탐욕적 선택 중 일부는 최적해에 포함이 안 될 수도 있었음



동전의 개수: 8개

남은 금액: 0원



다행히 다른 조합이 없기 때문에,

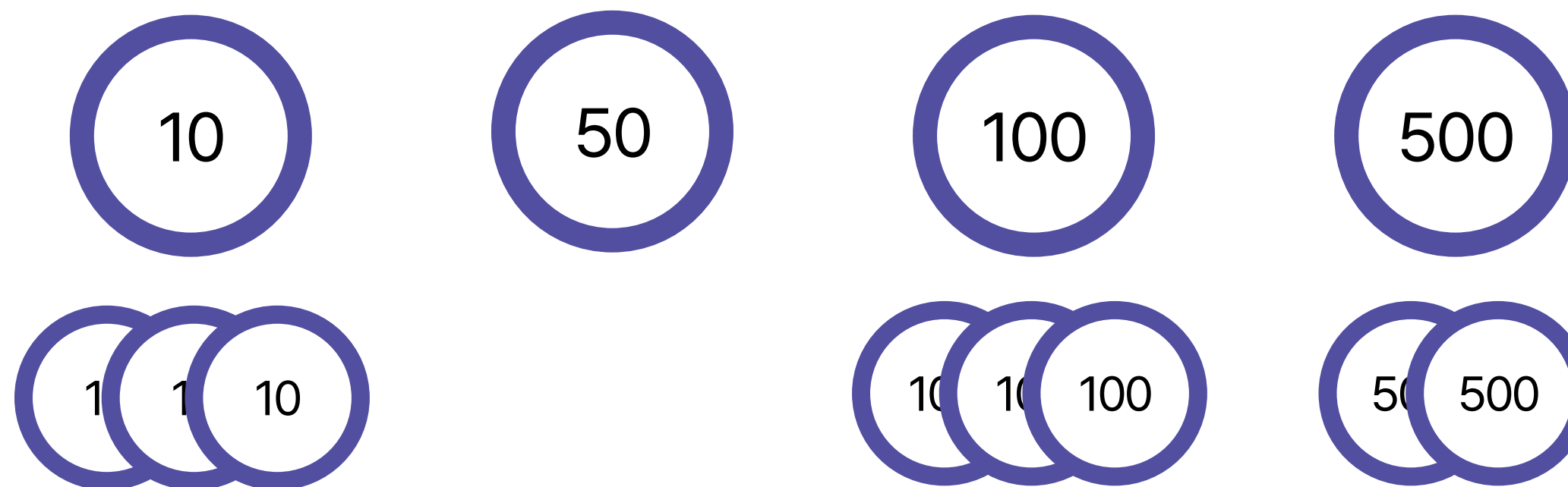
우리는 각각의 선택이 전부 최적해에 포함된다고 할 수 있음



탐욕적 선택 조건

동전의 개수: 8개

남은 금액: 0원



이런 조건을 탐욕적 선택 조건이라고 부름



결국, 어떤 문제가 그리디로 해결되는지 판단하기 위해선,
최적 부분 구조와 탐욕적 선택 조건을 만족하는지 확인해야 함



그리디를 적용할 수 있는 조건 2가지

- **최적 부분 구조**(optimal substructure)
 - 전체 문제의 최적의 해결 방법은 각 부분 문제의 최적 해결 방법의 조합으로 이루어 진다
- **탐욕적 선택 속성**(greedy choice property)
 - 현재 상황에서의 최적 선택이 항상 전체 문제의 최적해로 이어진다